Advanced Database System



Chapter 1

“Transaction Management and Concurrency Control ”

Melaku M.

Target Group – G3 Software Engineering, Section A & B

Learning Objectives

- **After completing this chapter, you will be able to:**
  - Describe the database transaction management process
  - Identify the four properties of a database transaction
  - Explain Concept of Serializability
  - Explain the problems of Concurrent Sharing
  - Describe how locking methods are used for concurrency control
  - Describe how stamping methods are used for concurrency control
  - Describe the role of database recovery management in maintaining database integrity

Transaction

- A transaction is a sequence of **one or more database operations** performed as a single, indivisible unit of work.
- A transaction is a logical unit of work that must be entirely completed or entirely aborted; no intermediate states are acceptable.
- A successful transaction changes the database from one consistent state to another. A consistent database state is one in which all **data integrity constraints** are satisfied.

Transaction concepts

- **Example**: Transaction to transfer $50 from account A to account B

1. read(A)

2. A := A – 50

3. write(A)

4. read(B)

5. B := B + 50

6. write(B)

The **DBMS** ensures that all six steps are executed as a single unit of work. If any step fails, the entire transaction is rolled back, and the database remains in a consistent state.

Cont'd

Example: Transaction to transfer $50 from account A to account B:

1. read(A)

2. A := A – 50

3. write(A)

4. read(B)

5. B := B + 50

6. write(B)

- If the transaction fails after **step 3 and before step 6**, money will be "lost" leading to an inconsistent database state, which is not acceptable by the bank.
  - Failure could be due to software or hardware
- To solve this problem, we have two important operations:
  - ✓ **Commit**: It is used to save the work done permanently.
  - ✓ **Rollback**: It is used to undo the work done.

Access to the DB is done by two operation

- **Read(X):** Read operation is used to read the value of X from the database and stores it in a buffer in main memory.
- **Write(X):** Write operation is used to write the value back to the database from the buffer.

Transaction Management with SQL

- ANSI has defined standards that govern SQL database transactions. Transaction support is provided by two SQL statements: COMMIT and ROLLBACK.
- A **COMMIT** statement is reached, in which case all changes are permanently recorded within the database. The COMMIT statement automatically ends the SQL transaction.
- A **ROLLBACK** statement is reached, in which case all changes are aborted and the database is rolled back to its previous consistent state.

ACID Properties: transaction has the four properties

- To preserve the integrity of data the database system must ensure:
  1. Atomicity
  2. Consistency
  3. Isolation
  4. Durability
- The ACID properties are essential for maintaining the integrity, reliability, and consistency of transactions in a database, particularly in environments where multiple transactions occur simultaneously.

1. Atomicity

- A transaction is treated as a single, indivisible unit of work. **It either completes in full or not at all.** If any part of the transaction fails, the entire transaction is rolled back, leaving the database in its previous state(unchanged).
- If a transaction T1 has four operation, all four operation must be successfully completed; otherwise, the entire transaction is aborted.
- This ensures that the database remains consistent.

2. Consistency

- **A transaction must preserve/follow the database's integrity constraints.**
- A transaction takes/brings a database from one consistent state/valid to another. When a transaction is completed, the database must be in a consistent state. If any of the transaction parts violates an integrity constraint or business rules, the entire transaction is aborted.
- The execution of a transaction will leave a database in either its prior stable state or a new stable state.

3. Isolation

- **Concurrent transactions should not interfere with each other.**
- Isolation means that the data used during the execution of first transaction cannot be used by a second transaction until the first one is completed.
- In other words, if transaction T1 is being executed and is using the data item X, that data item cannot be accessed by any other transaction (T2 … Tn) until T1 ends.
- This prevents anomalies like lost updates, dirty reads, & non-repeatable reads.
- This property is particularly useful in multiuser database environments because several users can access and update the database at the same time.

Durability

- Ensures that once transaction changes are done and committed, they cannot be undone or lost, even in the event of a system failure.

Example: Atomicity

Transaction to transfer $50 from account A to account B:

1. **read**(*A*)
2. *A* := *A* – 50
3. **write**(*A*)
4. **read**(*B*)
5. *B* := *B* + 50
6. **write**(*B*)

Note: don't focus on commit and abort

Atomicity requirement

- If the transaction fails after step 3 and before step 6, money will be "lost" leading to an inconsistent database state
  - Failure could be due to software or hardware
- The system should ensure that updates of a partially executed transaction are not reflected in the database

Example of Fund Transfer : Concurrent Transactions

- **Isolation requirement:** If between steps 3 and 6, another transaction T2 is allowed to access the partially updated database, it will see an inconsistent database (the sum A + B will be less than it should be).

T1	T2
1. **read**(*A*)	
2. *A* := *A* – 50	
3. **write**(*A*)	
	read(A), read(B), print(A+B)
4. **read**(*B*)	
5. *B* := *B* + 50	
6. **write**(*B*	

States of Transaction

State	Description
Active	Transaction is in progress.
Partially Committed	Transaction has finished executing but is not yet committed.
Failed	Transaction encountered an error and cannot proceed.
Aborted	Transaction has been rolled back and changes undone.
Committed	Transaction has been successfully completed and changes are permanent.

Concurrent Execution

- **Concurrent Execution**: multiple transactions run concurrently.
- **Serial Execution**—that is, one transaction execute at a time, each starting only after the previous one has completed. It ensures **isolation** property by default.
- When several transactions run concurrently, the **isolation property** may be violated, resulting in database consistency being destroyed despite the correctness of each individual transaction.
- **Concurrent execution violates the isolation properties and causes the following problems** – lost update, dirty read, and unrepeated read.

Concurrent Executions(cont'd)

- Multiple transactions are allowed to run concurrently in the system.
- **Benefits are:**
  - ✓ **Increased processor and disk utilization**, leading to better transaction throughput
    - ✓ e.g., one transaction can be using the CPU while another is reading from or writing to the disk.
  - ✓ **Reduced average response time for transactions:** short transactions need not wait behind long ones. This increase the transaction throughput.
- **Concurrency control schemes** – mechanisms to achieve isolation
  - That is, to control the interaction among the concurrent transactions in order to prevent them from destroying the consistency of the database.

Problems with Concurrent Execution

- Concurrent/Simultaneous execution of transactions over a shared database can create several **data integrity** and consistency problems.
- The three main problems are:
  1. Lost updates,
  2. Uncommitted data, and
  3. Inconsistent retrievals.

A. Lost Updates

- The lost update problem occurs when two concurrent transactions, T1 and T2, are updating the same data element and one of the updates is lost (overwritten by the other transaction).
- For example: Consider the diagram here, where two transactions TX and TY, are performed on the same account A where the balance of account A is $300.

Time	T_x	T_y
t_1	READ (A)	—
t_2	A = A - 50	
t_3	—	READ (A)
t_4	—	A = A + 100
t_5	—	—
t_6	WRITE (A)	—
t_7		WRITE (A)

LOST UPDATE PROBLEM

A. Lost Updates

- At time t1, transaction T_X reads the value of account A, i.e., $300 (only read).
- At time t2, transaction T_X deducts $50 from account A that becomes $250 (only deducted and not updated/write).
- Alternately, at time t3, transaction T_Y reads the value of account A that will be $300 only because T_X didn't update the value yet.
- At time t4, transaction T_Y adds $100 to account A that becomes $400 (only added but not updated/write).
- At time t6, transaction T_X writes the value of account A that will be updated as $250 only, as T_Y didn't update the value yet.
- Similarly, at time t7, transaction T_Y writes the values of account A, so it will write as done at time t4 that will be $400. It means the value written by T_X is lost, i.e., $250 is lost.

Hence data becomes incorrect, and database sets to inconsistent.

B. Dirty Read

- Also, known as **uncommitted data**, occurs when two transactions, T1 and T2, are executed concurrently and the first transaction (T1) is rolled back, because of server down/transaction failure after the second transaction (T2) has already accessed the uncommitted data

Time	T_x	T_y
t_1	READ (A)	—
t_2	A = A + 50	—
t_3	WRITE (A)	—
t_4	—	READ (A)
t_5	SERVER DOWN ROLLBACK	—

DIRTY READ PROBLEM

Cont'd

- At time t1, transaction T_X reads the value of account A, i.e., $300.
- At time t2, transaction T_X adds $50 to account A that becomes $350.
- At time t3, transaction T_X writes the updated value in account A, i.e., $350.
- Then at time t4, transaction T_Y reads account A that will be read as $350.
- Then at time t5, transaction T_X rollbacks due to server problem, and the value changes back to $300 (as initially).
- But the value for account A remains $350 for transaction T_Y as committed, which is the dirty read and therefore known as the Dirty Read Problem.

C. Unrepeatable Reads

- Also known as **inconsistent Retrieval**.
- Inconsistent retrievals occur when a transaction accesses data before and after one or more other transactions finish working with such data.
  - *Occurs when two different values are read for the same database item by a transaction.*

C. Inconsistent Retrieval

- For example: Consider two transactions, TX and TY, performing the read/write operations on account A, having an available balance = $300.

Time	T_x	T_y
t_1	READ (A)	—
t_2	—	READ (A)
t_3	—	A = A + 100
t_4	—	WRITE (A)
t_5	READ (A)	—

UNREPEATABLE READ PROBLEM

Thus, in order to maintain consistency in the database and avoid such problems that take place in concurrent execution, management is needed, and that is where the concept of Concurrency Control comes into role .

Schedule

- **Schedule** – a sequences/arrangement of instructions that ***specify the chronological order in which instructions of concurrent transactions are executed.***
  - A schedule for a set of transactions must consist of all instructions of those transactions
  - Must preserve the order in which the instructions appear in each individual transaction.
- A transaction that successfully completes its execution will have a **commit** instructions as the last statement.
  - *By default transaction assumed to execute commit instruction as its last step.*
- A transaction that fails to successfully complete its execution will have an **abort** instruction as the last statement.

Consider the simplified banking system

- Let T1 and T2 be two transactions that transfer funds from one account to another.

- Transaction T1 transfers $50 from account A to account B. It is defined as:

```
T1: read(A);
    A := A − 50;
    write(A);
    read(B);
    B := B + 50;
    write(B).
```

Transaction T2 transfers 10 percent of the balance from account A to account B. It is defined as:

```
T2: read(A);
    temp := A * 0.1;
    A := A − temp;
    write(A);
    read(B);
    B := B + temp;
    write(B).
```

Schedule 1

Schedule 1—**a serial schedule** in which T1 is followed by T2

Initially A=$1000, B=$2000

T_1	T_2
read (A)	
$A := A - 50$	
write (A)	
read (B)	
$B := B + 50$	
write (B)	
commit	
	read (A)
	$temp := A * 0.1$
	$A := A - temp$
	write (A)
	read (B)
	$B := B + temp$
	write (B)
	commit

Serial Schedules:

One transaction executed after the another, no mixed execution(T1 with T2).

We know that each transaction, run in isolation, will preserve consistency.

The total amount of money in accounts A and B=3000

Schedule 2

Schedule 2—a serial schedule in which T2 is followed by T1.

T_1	T_2
	read(A)
	$temp := A * 0.1$
	$A := A - temp$
	write(A)
	read(B)
	$B := B + temp$
	write(B)
read(A)	
$A := A - 50$	
write(A)	
read(B)	
$B := B + 50$	
write(B)	

A:=900

A:=850

B:=2100

B:=2150

A: = 1000

100

A: = 900

B: = 2000

B: = 2100

These schedules are serial:

A+B: = 850+2150 = 3000

Schedule 3

Let T1 and T2 be the transactions defined previously. The following schedule is **not a serial schedule**, but it is **equivalent** to Schedule 1.

Schedule 3—a concurrent schedule(serializable schedule) equivalent to schedule 1.

- In this schedule, we see that the effect of the two transactions scheduled in this manner is the same as for the serial schedule 1.

	T_1	T_2	
A:=1000	read(A)		
	$A := A - 50$		
A:=950	write(A)		
		read(A)	A:=950
		$temp := A * 0.1$	
		$A := A - temp$	
A:=855		write(A)	
B:=2000	read(B)		
	$B := B + 50$		
B:=2050	write(B)		
		read(B)	
		$B := B + temp$	
B:=2145		write(B)	

In Schedules 1, 2 and 3, the sum A + B is preserved.

Schedule 4

- The following concurrent schedule does **not preserve** the value of (A + B).

Schedule 4—a concurrent schedule resulting in an inconsistent state.

Not serializable schedule

Not all concurrent executions result in a correct state.

	T_1	T_2	
A:=950	read(A)		
	$A := A - 50$		
		read(A)	A:=1000
		$temp := A * 0.1$	
		$A := A - temp$	
		write(A)	
		read(B)	A:=900
	write(A)		B:=2000
A:=900	read(B)		
B:=2000	$B := B + 50$		
	write(B)		
	commit		
B:=2050		$B := B + temp$	
		write(B)	
		commit	B:=2150

A+B: = 900+2150 = 3050

Simplified view of transactions

- We ignore operations other than read and write instructions
- We assume that transactions may perform arbitrary computations on data in local buffers in between reads and writes.
- Our simplified schedules consist of only **read** and **write** instructions.

T_1	T_2
read(A)	
write(A)	
	read(A)
	write(A)
read(B)	
write(B)	
	read(B)
	write(B)

Schedule 3— showing only the read and write instructions.

Conflicting Instructions

- Instructions **li** and **lj** of transactions **Ti** and **Tj** respectively, **conflict** if and only if there exists some **item Q** accessed by both li and lj, and at least one of these instructions wrote Q.
    1. li = **read**(Q), lj = **read**(Q). **They don't conflict.**
    2. li = **read**(Q), lj = **write**(Q). **They conflict.**
    3. li = **write**(Q), lj = **read**(Q). **They conflict.**
    4. li = **write**(Q), lj = **write**(Q). **They conflict.**

RR; Neither transaction can affect the other's results, so no anomaly arises.

- If li and lj are **consecutive** in a schedule and they do not conflict, their results would remain the same even if they had been interchanged in the schedule.

Conflicting Instructions

- Let us consider a schedule S in which there are two consecutive instructions, I and J, of transactions Ti and Tj , respectively ($i \neq j$).
- If I and J refer to different data items, then we can swap order of I and J without affecting the results of any instruction in the schedule. However, if I and J refer to the same data item Q, then the order of the two steps may matter.
- Since the write(A) instruction of T2 in schedule 3, does not conflict with the read(B) instruction of T1, we can swap these instructions to generate an equivalent schedule, schedule 5. Regardless of the initial system state, schedules 3 and 5 both produce the same final system state.

Conflicting Instructions

Schedule 5— schedule 3 after swapping of a pair of instructions.

T_1	T_2
read(A)	
write(A)	
	read(A)
read(B)	
	write(A)
write(B)	
	read(B)
	write(B)

We continue to swap nonconflicting instructions:

- Swap the read(B) instruction of T_1 with the read(A) instruction of T_2.
- Swap the write(B) instruction of T_1 with the write(A) instruction of T_2.
- Swap the write(B) instruction of T_1 with the read(A) instruction of T_2.

Conflicting Instructions

Schedule 6—a serial schedule that is equivalent to schedule 3.

T_1	T_2
read(A)	
write(A)	
read(B)	
write(B)	
	read(A)
	write(A)
	read(B)
	write(B)

The final result of these swaps, schedule 6 is a serial schedule. Note that **schedule 6** is exactly the same as **schedule 1**, but it shows only the read and write instructions. Thus, we have shown that **schedule 3** is equivalent to a **serial schedule**. This equivalence implies that, regardless of the initial system state, schedule 3 produces the same final state as some serial schedule.

Serializability

- ***A (possibly concurrent) schedule is Serializable if it is equivalent to a serial schedule.***
- Serializability: A property in which the selected order of concurrent transaction operations creates the same final database state that would have been produced if the transactions had been executed in a serial fashion.
- Different forms of **schedule equivalence** give rise to the notions of:
    1. conflict serializability
    2. view serializability

Conflict Serializability

- If a schedule S can be transformed into **a schedule S'** by a series of swaps of non-conflicting instructions, we say that S and S' are **conflict equivalent**.
- We say that a schedule S is **conflict serializable** if it is conflict equivalent to a serial schedule.
- If there is cycle in the schedule it is **not-conflict equivalent** and **conflict serializable**.
- Thus, schedule 3 is **conflict serializable**, since it is **conflict equivalent** to the serial schedule 1.

Conflict Serializability(Cont'd)

- Schedule 3 can be transformed into Schedule 6, **a serial schedule** where T2 follows T1, by series of swaps of non-conflicting instructions.
- Therefore Schedule 3 is **conflict serializable.**

T_1	T_2
read (A)	
write (A)	
	read (A)
	write (A)
read (B)	
write (B)	
	read (B)
	write (B)

Schedule 3



T_1	T_2
read (A)	
write (A)	
read (B)	
write (B)	
	read (A)
	write (A)
	read (B)
	write (B)

Schedule 6

Conflict Serializability(Cont'd)

- Example of a schedule that is **not conflict serializable**:

T_3	T_4
read (Q)	
	write (Q)
write (Q)	

- We are unable to swap instructions in the above schedule to obtain either the serial schedule **< T3, T4 >,** or the serial schedule **< T4, T3 >.**

Conflict Serializability(Cont'd)

Is this schedule conflict serializable or not?

T_1	T_5
read(A)	
$A := A - 50$	
write(A)	
	read(B)
	$B := B - 10$
	write(B)
read(B)	
$B := B + 50$	
write(B)	
	read(A)
	$A := A + 10$
	write(A)

Figure 17.13 Schedule 8.

View Serializability

- However, there are other definitions of **schedule equivalence** based purely on the read and write operations.
- One such definition is **view equivalence**, a definition that leads to the concept of **view serializability**. View serializability is not used in practice due to its high degree of computational complexity.

View Serializability

- Let *S* and *S'* be two schedules with the same set of transactions. *S* and *S'* are **view equivalent** if the following three conditions are met, for each data item *Q*,
  1. If in schedule S, transaction T_i reads the initial value of *Q*, then in schedule *S'* also transaction T_i must read the initial value of *Q*.
  2. If in schedule S transaction T_i executes **read**(*Q*), and that value was produced by transaction T_j (if any), then in schedule *S'* also transaction T_i must read the value of *Q* that was produced by the same **write**(Q) operation of transaction T_j .
  3. The transaction (if any) that performs the final **write**(*Q*) operation in schedule *S* must also perform the final **write**(*Q*) operation in schedule *S'*.
- As can be seen, view equivalence is also based purely on **reads** and **writes** alone.

Concurrency Control

- How to assure that concurrently executing transactions preserve correctness of the database state? The abstract requirement is called **serializability**, and there is an important, stronger condition called conflict-serializability that most **schedulers** actually enforce.
- The objective of concurrency control is to ensure the **serializability of transactions** in a multiuser database environment. To achieve this goal, most concurrency control techniques are oriented toward preserving the **isolation** property of concurrently executing transactions.
- Concurrency control is important because the simultaneous execution of transactions over a shared database can create several data integrity and consistency problems. The three main problems are lost updates, uncommitted data, and inconsistent retrievals.

Concurrency Control

- There are variety of concurrency control schemes
  - ✓ Lock-Based Protocols
  - ✓ Timestamp-Based Protocols
  - ✓ Validation-Based Protocols

Lock-Based Protocols

- One way to ensure isolation is to require that data items be accessed in a **mutually exclusive** manner; that is, while one transaction is accessing a data item, no other transaction can modify that data item.
- A transaction can access a data item only if it is currently holding a lock on that item.
- A lock guarantees **exclusive use** of a data item to a current transaction. A transaction acquires a lock prior to data access; the lock is released (unlocked) when the transaction is completed so that another transaction can lock the data item for its exclusive use.

Lock-Based Protocols

- A lock is a mechanism to control concurrent access to a data item
- Data items can be locked in two modes:
  1. **Shared**. If a transaction Ti has obtained a ***shared-mode lock (denoted by S)*** on item Q, then Ti can read, but cannot write, Q.
  2. **Exclusive**. If a transaction Ti has obtained an ***exclusive-mode lock (denoted by X)*** on item Q, then Ti can both read and write Q.
- Lock requests are made to concurrency-control manager. Transaction can proceed only after request is **granted**.

Lock-Based Protocols

❖ Lock-compatibility matrix

- A transaction may be granted a lock on an item if the requested lock is compatible with locks already held on the item by other transactions
- Any number of transactions can hold shared locks on an item, that means shared mode is compatible with shared mode, but not with exclusive mode.
- But if any transaction holds an exclusive on the item no other transaction may hold any lock on the item.

	S	X
S	true	false
X	false	false

Lock-Based Protocols

- When a transaction Ti requests a lock on a data item Q in a particular mode M, the concurrency-control manager **grants** the lock provided that:
  - ✓ There is no other transaction holding a lock on Q in a mode that conflicts with M.
  - ✓ There is no other transaction that is waiting for a lock on Q and that made its lock request before Ti.

Schedule With Lock Grants

- Let A and B be two accounts that are accessed by transactions T1 and T2. Transaction T1 transfers $50 from account B to account A (Figure 18.2). Transaction T2 displays the total amount of monev in accounts A and B—that is, the sum A + B (Figure18.3).

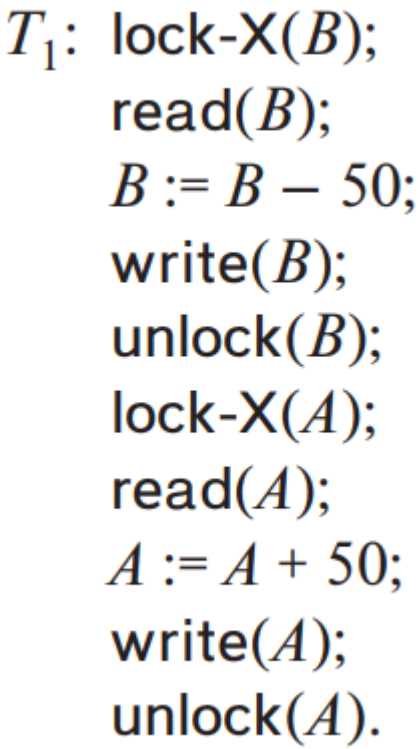


Figure 1.2 .2 Transaction T1.

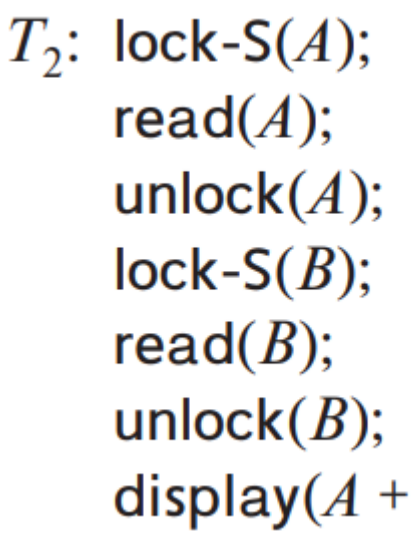


Figure 1.2 .2 Transaction T2.

- This schedule is not serializable (why?).
- Each transaction in the system follow a set of rules, called a locking protocol, indicating when a transaction may lock and unlock each of the data items.
- Locking protocols enforce serializability by **restricting** the set of possible schedules.

Grants omitted in rest of chapter
lock must be granted in the interval of time

T_1	T_2	concurrency-control manager
lock-X(B)		
		grant-X(B, T_1)
read(B)		
$B := B - 50$		
write(B)		
unlock(B)		
	lock-S(A)	
		grant-S(A, T_2)
	read(A)	
	unlock(A)	
	lock-S(B)	
		grant-S(B, T_2)
	read(B)	
	unlock(B)	
	display($A + B$)	
lock-X(A)		
		grant-X(A, T_1)
read(A)		
$A := A + 50$		
write(A)		
unlock(A)		

Schedule 1.

Cont'd

- Suppose that the values of accounts A and B are $100 and $200, respectively. If these two transactions are executed serially, either in the order T1, T2 or the order T2, T1, then transaction T2 will display the value $300.
- If, however, these transactions are executed concurrently, then schedule 1, is possible. In this case, transaction T2 displays $250, which is incorrect. The reason for this **mistake** is that the **transaction T1 unlocked data item B too early, as a result of which T2 saw an inconsistent state.**

Cont'd

- Suppose now that unlocking is delayed to the end of the transaction. Transaction T3 corresponds to T1 with unlocking delayed. Transaction T4 corresponds to T2 with unlocking delayed.

Schedule 1 has printed inconsistent total. But other schedules are possible, which prints consistent total. T4 will not print out an inconsistent result in any of them; we shall see why later.

T_3: lock-X(B);
read(B);
$B := B - 50$;
write(B);
lock-X(A);
read(A);
$A := A + 50$;
write(A);
unlock(B);
unlock(A).

Transaction T3

T_4: lock-S(A);
read(A);
lock-S(B);
read(B);
display($A + B$);
unlock(A);
unlock(B).

Transaction T4

Cont'd

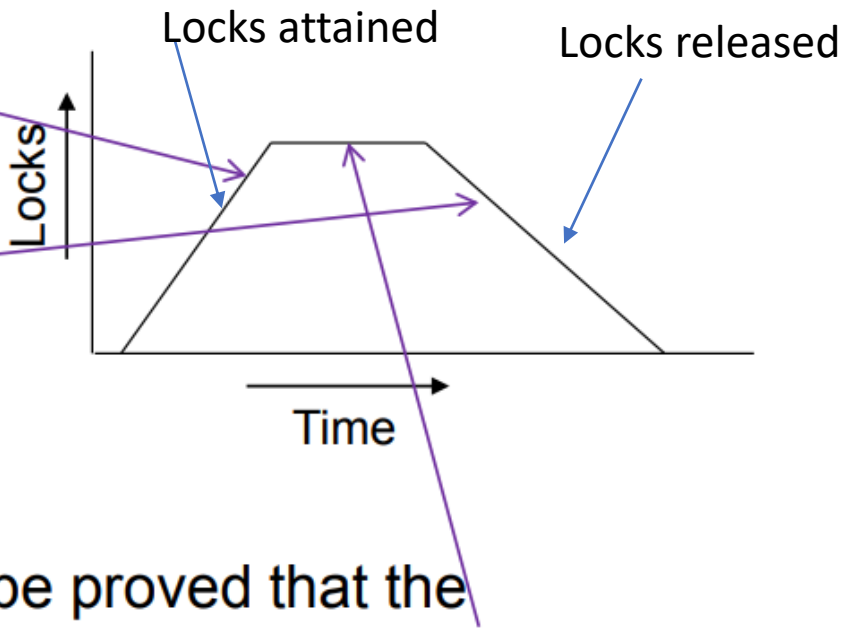
- Unfortunately, locking can lead to an undesirable situation. Consider the partial schedule of Figure 18.7 for T3 and T4. Since T3 is holding an exclusive-mode lock on B and T4 is requesting a shared-mode lock on B, T4 is waiting for T3 to unlock B.
- Similarly, since T4 is holding a shared-mode lock on A and T3 is requesting an exclusive mode lock on A, T3 is waiting for T4 to unlock A. Thus, we have arrived at a state where neither of these transactions can ever proceed with its normal execution. This situation is called **deadlock**.
- When deadlock occurs, the system must roll back one of the two transactions.

T_3	T_4
lock-X(B)	
read(B)	
$B := B - 50$	
write(B)	
	lock-S(A)
	read(A)
	lock-S(B)
lock-X(A)	

Schedule 2.

Lock –based protocols: The Two-Phase Locking Protocol

- A protocol which ensures conflict-serializable schedules.
- Phase 1: **Growing Phase**
  - Transaction may obtain locks
  - Transaction may not release locks
- Phase 2: **Shrinking Phase**
  - Transaction may release locks
  - Transaction may not obtain locks



- The protocol assures serializability. It can be proved that the transactions can be serialized in the order of their **lock points** (i.e., the point where a transaction acquired its final lock)(the end of its growing phase)

❖ This protocol requires that each transaction issue **lock** and **unlock requests** in two phases.

Lock –based protocols: The Two-Phase Locking Protocol

- For example, in **schedule 2** transactions T3 and T4 are two phase. On the other hand, in **schedule 1** transactions T1 and T2 are not two phase.
- Two-phase locking guarantees serializability, but it does not prevent deadlocks.
- Observe that transactions T3 and T4 are two phase, but, in schedule 2, they are deadlocked.

Lock –based protocols: The Two-Phase Locking Protocol

- In addition to being serializable, schedules should be cascadeless.
- **Cascading rollback** may occur under two-phase locking. It occurs because of **dirty read.**
- As an illustration, consider the partial schedule of Figure 18.8.
- Each transaction observes the two-phase locking protocol, but the failure of T5 after the read(A) step of T7(at time t_{12}) leads to cascading rollback of T6 and T7.

Cascading rollback/reaction of chain rollback occurs when all transaction are not committed.

T_5	T_6	T_7
lock-X(A)		
read(A)		
lock-S(B)		
read(B)		
write(A)		
unlock(A)		
	lock-X(A)	
	read(A)	
	write(A)	
	unlock(A)	
		lock-S(A)
		read(A)

Figure 18.8 Partial schedule under two-phase locking.

Lock – based protocols: Strict two-phase locking

- **Cascading rollbacks** can be avoided by a modification of two-phase locking called the strict two-phase locking protocol.
- All **exclusive-mode locks** taken by a transaction be held until that transaction commits.
  - ✓ That means, exclusive locks held by a transaction are released only after it has successfully committed or aborted.
- This requirement ensures that any data written by an uncommitted transaction are locked in **exclusive mode** until the transaction commits, preventing any other transaction from reading the data.

Lock –based protocols: Rigorous two-phase locking

- Another variant of two-phase locking is the **rigorous two-phase locking protocol**, which requires that **all locks be held until the transaction commits**.
- We can easily verify that, with rigorous two-phase locking, transactions can be serialized in the order in which they commit.

Strict phase locking: transaction must hold all its exclusive locks till it commits/aborts.

Strict 2PL	
T1	T2
s-lock(A)	
read(A)	
	s-lock(A)
x-lock(B)	
unlock(A)	
read(B)	
write(B)	
	read(A)
	unlock(A)
commit	
unlock(B)	
	s-lock(B)
	read(B)
	unlock(B)
	commit

Rigorous 2PL: more stricter: all locks are held till commit/abort.

Rigorous 2PL	
T1	T2
s-lock(A)	
read(A)	
	s-lock(A)
x-lock(B)	
	read(A)
read(B)	
write(B)	
commit	
unlock(B)	
	s-lock(B)
	read(B)
unlock(A)	
	commit
	unlock(A)
	unlock(B)

–It is used in dynamic environments where

lock conversions

Two-phase locking protocol with lock conversions:

– In Growing Phase:

- can acquire a lock-S on item
- can acquire a lock-X on item
- can convert a lock-S to a lock-X (upgrade)

– In Shrinking Phase:

- can release a lock-S
- can release a lock-X
- can convert a lock-X to a lock-S (downgrade)

T_8	T_9
lock-S(a_1)	
	lock-S(a_1)
lock-S(a_2)	
	lock-S(a_2)
lock-S(a_3)	
lock-S(a_4)	
	unlock(a_1)
	unlock(a_2)
lock-S(a_n)	
upgrade(a_1)	

Implementation of Locking

- A **lock manager** can be implemented as a separate process
- Transactions can send lock and unlock requests as messages
- The lock manager replies to a lock request by sending a lock grant messages (or a message asking the transaction to roll back, in case of a deadlock)
- The requesting transaction waits until its request is answered.
- The lock manager maintains an in-memory data-structure called a **lock table** to record granted locks and pending requests

Timestamp-Based Protocols

- Each transaction Ti enters the system, we associate a **unique fixed timestamp**, denoted by TS(Ti). This timestamp is assigned by the DBMS before the transaction Ti starts execution.
- If a transaction Ti has been assigned timestamp TS(Ti), and a new transaction Tj enters the system, then TS(Ti) < TS(Tj).
- To determine the **timestamp** of the transaction, this timestamp-based protocol uses system Time(value of the **system clock)** or logical counter.
- Timestamp-based protocols manage concurrent execution such that time-stamp order = serializability order

Timestamp-Ordering Protocol(TSO)

- The timestamps of the transactions determine the **serializability order**. Thus, if TS(Ti) < TS(Tj), then the system must ensure that the **produced schedule** is equivalent to a serial schedule in which transaction Ti appears before transaction Tj.
- We associate with each data item Q, two TS values:
  1. **W-timestamp(Q):** is the largest TS of any transaction that executed **write**(Q) successfully.
  2. **R-timestamp(Q):** is the largest TS of any transaction that executed **read**(Q) successfully.
- These timestamps are updated whenever a new read(Q) or write(Q) instruction is executed.

Timestamp-Ordering Protocol

- The timestamp-ordering protocol **ensures** that any conflicting read and write operations are executed in timestamp order.
- Suppose a transaction Ti issues a read(Q):
  1. **If TS(Ti) < W-timestamp(Q),** then Ti needs to read a value of Q that was already overwritten.
     - Hence, the read operation is rejected, and Ti is rolled back.

  2. If TS(Ti) ≥ W-timestamp(Q), then the read operation is executed, and R-timestamp(Q) is set to max(R-timestamp(Q), TS(Ti)).

Timestamp-Ordering Protocol

- Suppose that transaction Ti issues write(Q).

1. If TS(Ti) < R-timestamp(Q), then the value of Q that Ti is producing was needed previously, and the system assumed that that value would never be produced.
   - Hence, the system rejects the write operation and rolls Ti back.
2. If TS(Ti) < W-timestamp(Q), then Ti is attempting to write an obsolete value of Q.
   - Hence, the system rejects this write operation and rolls Ti back.
3. Otherwise, the system executes the write operation and sets W-timestamp(Q) to TS(Ti).

Timestamp-Ordering Protocol

- If a transaction Ti is rolled back by the concurrency-control scheme as result of issuance of either a read or write operation, the system assigns it a new timestamp and restarts it.
- To illustrate this protocol, we consider transactions T25 and T26.

T_{25}: read(B);
read(A);
display($A + B$).

T_{26}: read(B);
$B := B - 50$;
write(B);
read(A);
$A := A + 50$;
write(A);
display($A + B$).

Presenting schedules under the timestamp protocol, we shall assume that a transaction is assigned a timestamp immediately before its first instruction.

Example of Schedule Under TSO

- Is this schedule valid under TSO?
- Assume that initially: R-TS(A) = W-TS(A) = 0 R-TS(B) = W-TS(B) = 0 Assume TS(T25) = 25 and TS(T26) = 26

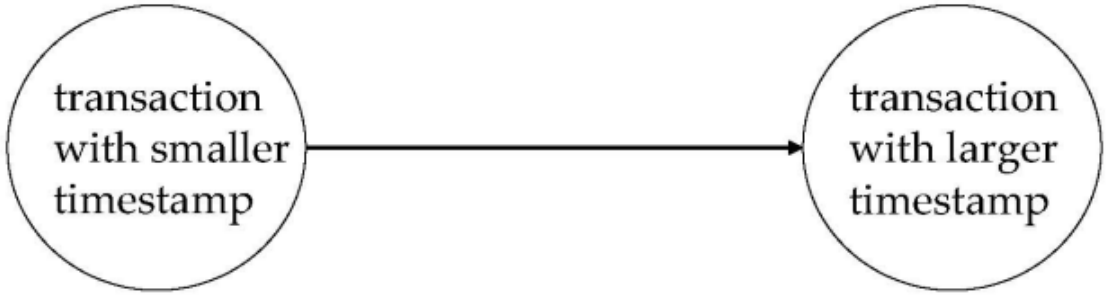
T_{25}	T_{26}
read(B)	
	read(B)
	$B := B - 50$
	write(B)
read(A)	
	read(A)
display($A + B$)	
	$A := A + 50$
	write(A)
	display($A + B$)

T_{27}	T_{28}
read(Q)	
	write(Q)
write(Q)	

And how about this one, where initially R-TS(Q)=W-TS(Q)=0

Correctness of Timestamp-Ordering Protocol

- The timestamp-ordering protocol ensures **conflict serializability**. This is because conflicting operations are processed in timestamp order, since all the arcs in the precedence graph are of the form.



Thus, there will be no cycles in the precedence graph

- Timestamp protocol ensures freedom from **deadlock** as no transaction ever waits. But the schedule may not be **cascade-free**, and may not even be recoverable.

Database Recovery

- Process of restoring database to a correct state in the event of a failure, data loss or corruption.
- **Type of Failure:**
  - System Crash:
  - Transaction Failure
  - Media Failure:
  - Natural physical disasters.
  - Sabotage.
  - viruses,
  - Incorrect command execution

Common strategies and methods for database recovery:

- DBMS should provide following facilities to assist with recovery:

1) **Backup mechanism**, which makes ***periodic backup copies*** of database.
2) **Replication,** Creating copies of the database on different servers to ensure availability
3) **Logging facilities**, which keep track of all changes made to the database during transactions. It's used to reconstruct the database state in case of a failure
4) **Checkpoint facility**, which enables updates to database in progress to be made permanent. Regular snapshots of the database and its transaction log.
5) **Recovery manager**, which allows DBMS to restore database to consistent state following a failure.

Transactions and Recovery

- Transactions represent basic unit of recovery.
- Recovery manager responsible for atomicity and durability.
  - ✓ If failure occurs between commit and database buffers being flushed to secondary storage then, to ensure durability, recovery manager has to **redo** (**rollforward**) transaction‘s updates.
  - ✓ If transaction had not committed at failure time, recovery manager has to undo(**rollback**) any effects of that transaction for atomicity.


Thank You!